第13章 紅黑樹：詳解

13-E01 黑高與高度的分母

推導

紅child版bh(20)=1、bh(10)=bh(30)=1、bh(NIL)=0；h=2，每條含root完整黑數2（20與末端NIL）。黑child版bh(20)=2、bh(10)=bh(30)=1、bh(NIL)=0，h仍2，但含root完整黑數3。

父的bh不含父卻包含child若它為黑；child自己的bh又排除child，所以child黑時$bh(child) = bh(parent) - 1$，child紅時相等。高度歸納：兩側各至少$2^{bh(x)-1} - 1$內部點，合計$2(2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$。Root-NIL路含B黑R紅，根及NIL黑、無相鄰紅，R≤B−1；h+1=B+R≤2B−1，得h≤2bh(root)。所以$n \geq 2^{h/2} - 1$，$h \leq 2\log_2(n+1)$。

結論與檢核

相同鍵與形狀可有不同合法黑高；路上黑點至少約一半，不能將不等號反向。完整圖中的所有NIL都須計入，而非只計內部黑點。

13-E02 舊父與 beta 的完整指標

推導

先保存p30、y20、β15；寫10.right=15、15.parent=10、20.parent=30、30.left=20、20.left=10、10.parent=20。10.left及20.right的外側子樹不動。Root分支改為20.parent=NIL、root=20，無30.left寫入，其餘相同。

旋轉前y原父就是x，需改的是x原父p指向y。若拿y原父x再指回x，可能造自連結或漏接外父。共享NIL.parent只代表刪除修正當前x空位置的上下文；若旋轉把另一個空β的parent寫x，可能讓之後取錯sibling，所以僅β非NIL時改其parent。

結論與檢核

中序仍α,10,15,20,γ，record資料不搬。每個非NIL child.parent都能反向核對；根新parent為NIL。

13-E03 內側插入的兩次旋轉

推導

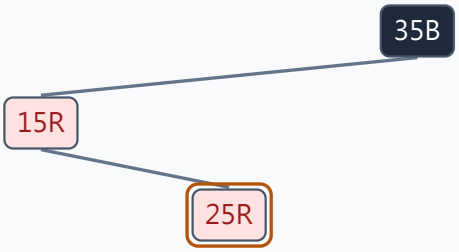
插35後35B；插15後35B-left15R；插25後15.right=25R，違規15R-25R，叔NIL黑。I2先z=舊p15、左旋15，得35B-left25R-left15R，算法z指15而非新插record25。I3把25黑、35紅，右旋35，得25B(left15R,right35R)。

第13章 紅黑樹：詳解 （續）

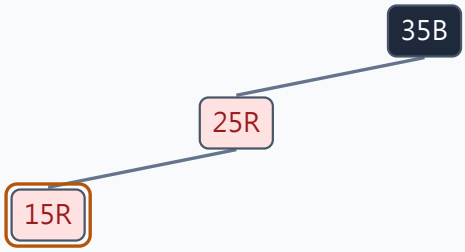
I2：左旋15，把內側折線拉成外側

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。

前：z=25，叔NIL黑



後：z=15，仍有紅紅

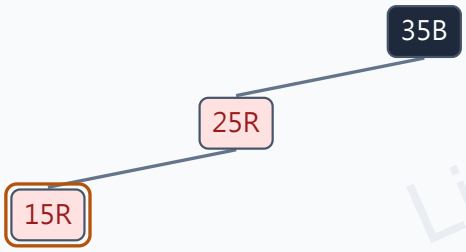


35仍是黑根；中序15,25,35；這一步必須接I3。

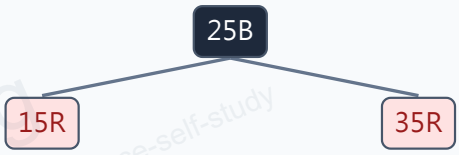
I3：25黑、35紅，再右旋35

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。

I3前：25/15紅紅



I3後：25黑，兩側紅



每條root-NIL含2黑；局部根色黑，不會與外父新造紅紅。

一般I2旋轉的兩個內部點皆紅，黑貢獻皆0，所以不改到邊界子樹的黑數。I3設四個黑根邊界子樹各完整黑數b，原gB下各路1+b，新pB下亦1+b；新gR的兩child黑，新zR父黑，消除唯一紅紅。這不依賴三鍵葉例的b=1。

結論與檢核

旋轉中心先15再35，總2次。I2後仍紅紅，必接I3；最後中序15,25,35、bh(root)=1。

13-E04 叔紅：完整黑數與 bh 不同

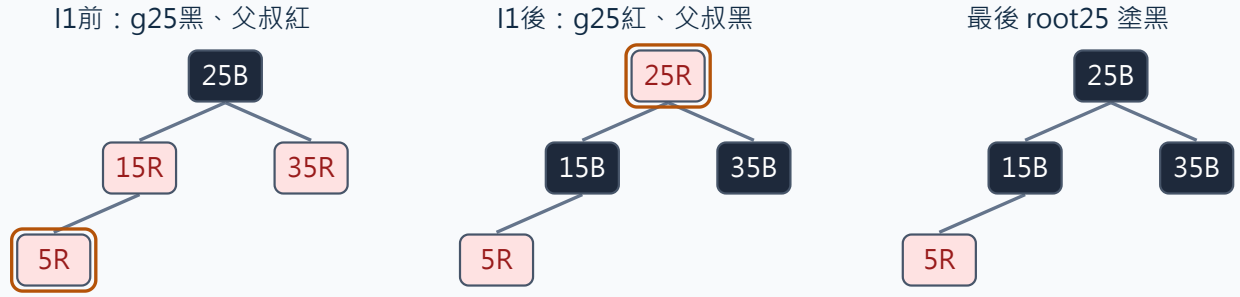
推導

I1前g25B、p15R、u35R、z5R。每條從g含自己的路到NIL黑數2，bh(g)=1。I1後15/35黑、25紅，算法z=25；完整黑數仍2，而bh(g)=2（排除紅g，包含一個黑child及NIL）。最後因g為根塗黑，完整黑數才升3，bh(g)仍2。

第13章 紅黑樹：詳解 （續）

I1 與根黑分開核對

B = 黑、R = 紅；橘框 = 本次迴圈z；最後根黑狀態不標z；未畫child為NIL黑。



完整黑數 2→2→3；bh(25) 1→2→2；非根g只做中間的守恆重上色。

若g非根，黑轉紅少1與p/u紅轉黑加1互抵，所有從祖先穿過g的完整黑數不變。唯一可能新違規是gR與g.parentR相鄰；設z=g把同一迴圈問題上移兩層。其他路及BST序未改。

結論與檢核

不能將bh(g)變大誤說成整棵樹所有路多一黑；只有根最後塗黑才是一致全樹增加。

13-E05 移後繼時保留 data

推導

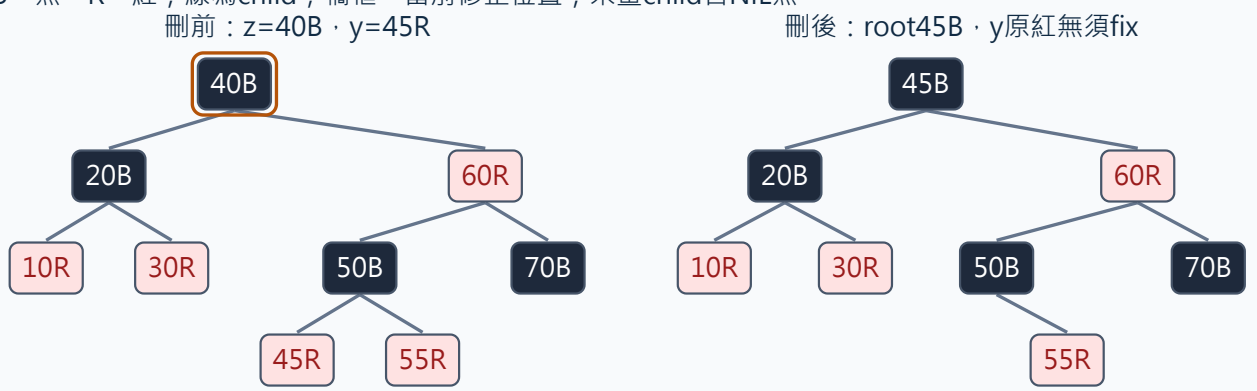
z40B；後繼y45R是50.left，x=y.right=NIL。保存yOriginalColor=R。TRANSPLANT(y,NIL)使50.left=NIL、NIL.parent=50；y.right=60、60.parent=45。TRANSPLANT(40,45)使root=45、45.parent=NIL；接45.left=20、20.parent=45，令45.color取40原黑。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第13章 紅黑樹：詳解 （續）

刪40：整個後繼45 record移到原根

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。



D45仍跟隨45；D40隨40刪除；中序只少40，其他record身分不變。

最後root45B，左20B(10R,30R)，右60R，其左50B(right55R)、右70B。Y現在黑是為了維持z原位置的黑數；缺黑判斷卻取原位置移走的y原色R，所以不做fix。原D45留在移動的45物件，原D40隨40移除，其餘record身分均保留。

結論與檢核

NIL.parent=50是此splice上下文；穩定操作結束後可重設，不能因此推所有空child的parent都是50。

13-E06 紅兄弟後接紅父吸收

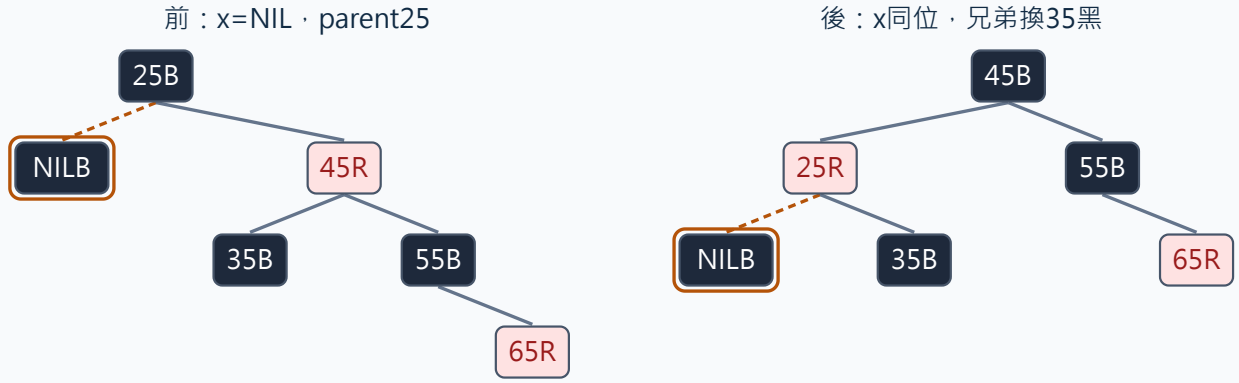
推導

刪前root25B、左15B、右45R(左35B、右55B(right65R))。Splice後15的位置成x=NIL，parent25、兄弟45R。D1把45黑、25紅，左旋25，得root45B，左25R(right35B)、右55B(right65R)，x仍25.left，新w35B。

第13章 紅黑樹：詳解 （續）

D1：紅兄弟45先變黑，左旋25

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。

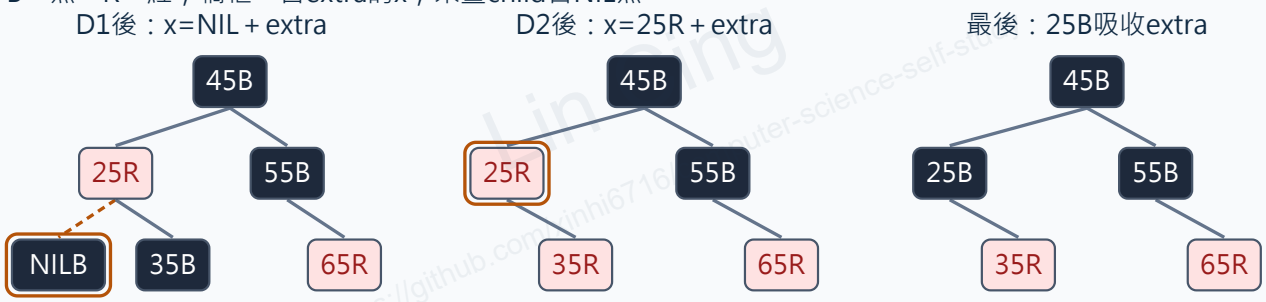


橘NIL含一份extra；轉換後extra尚在，接D2或D3/D4。

此時D2將35紅，x=25R；退出迴圈最後25塗黑，得45B-left25B(right35R)、right55B(right65R)。

D2 與最後塗黑：缺黑的移動及吸收分開畫

B = 黑、R = 紅；橘框 = 含extra的x；未畫child皆NIL黑。



D2：35變紅、x移到25；此時25仍紅。最後25塗黑，extra才消失。

令x實際黑數b、加extra為b+1。D1前p黑、w紅、w兩child各黑數b+1，所以含p路皆b+2；D1後新w黑，下方p紅與其兩路有效b+1，另一側亦b+1，總仍b+2。D2使p的兩分支實際黑數同為b，但少的黑上移p；p已因D1變紅，最後塗黑吸收，不再上升迭代。

結論與檢核

全過程只左旋25一次，沒有第二次D1。缺黑指的是25.left的位置，不能因共用NIL物件而丟掉此parent上下文。

第13章 紅黑樹：詳解 （續）

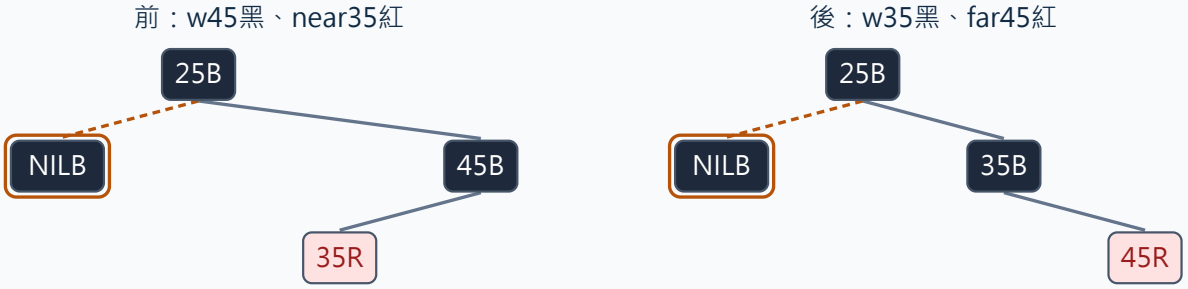
13-E07 近紅遠黑必須接 D4

推導

刪前25B-left15B、right45B(left35R)。Splice後p25B，x=NIL在left，w45B，near35R、far=NIL黑。D3將35黑、45紅，右旋45，得到25B-right35B(right45R)；x仍25.left，新的w35B、far45R。

D3：近端35紅，先右旋45

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。

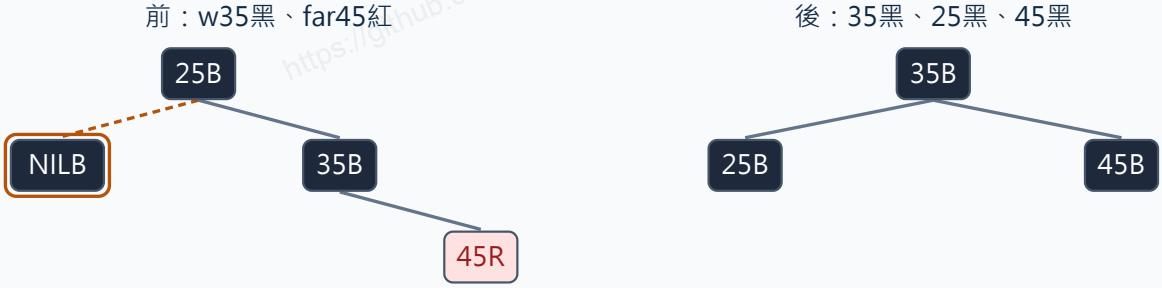


x=NIL仍是25.left，extra未消除；必接D4。

D4令35取25原黑、25黑、45黑，左旋25，結果35B(left25B,right45B)，extra消除。

D4：遠端45塗黑，左旋25完成

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。



三條key 25,35,45；每條root-NIL含3黑，不再有extra。

一般D4前含p的有效黑數c+b+1；變後w貢獻c、左pB的兩child實際b使左完整b+1，右far由紅轉黑亦b+1，故新總c+b+1相同。若w新紅，兩child都黑；外接根顏色仍等於原p色，不破壞上層紅黑關係。

結論與檢核

D3沒有消除extra，不能直接返回；兩次旋轉中心先45、再25，最後中序25,35,45且每條含根路3黑。

第13章 紅黑樹：詳解 （續）

13-E08 黑父 D2 與刪最後一點

推導

三黑點樹原bh(root25)=2。刪15後x=NIL在25.left，w35B兩child黑；D2將35紅、x上移root25B，迴圈因x=root停止，根的extra從全部路一起丟掉。結果25B-right35R，bh(root)=1，兩條root-NIL路仍同黑數。

D2 根吸收：整樹黑高可同降一

B = 黑、R = 紅；線為child；橘框 = 當前修正位置；未畫child皆NIL黑。



x由NIL上移root25，extra從所有根葉路一起去除；bh(root)2→1。

唯一黑根25時，x=25.right=NIL；TRANSPLANT將root=NIL、NIL.parent=舊25.parent=NIL。Y原黑仍進fix，但x=root，迴圈不執行，最後x.color=B（仍是NIL黑）。鍵集合空、bh(NIL)=0、h=0，五性質成立。

結論與檢核

刪除可令全樹黑高同降一；保持相等並不要求整段操作的黑高數值不變。NIL仍須黑，不能因空樹而漏掉此基例。

13-E09 右側缺黑與直接後繼

推導

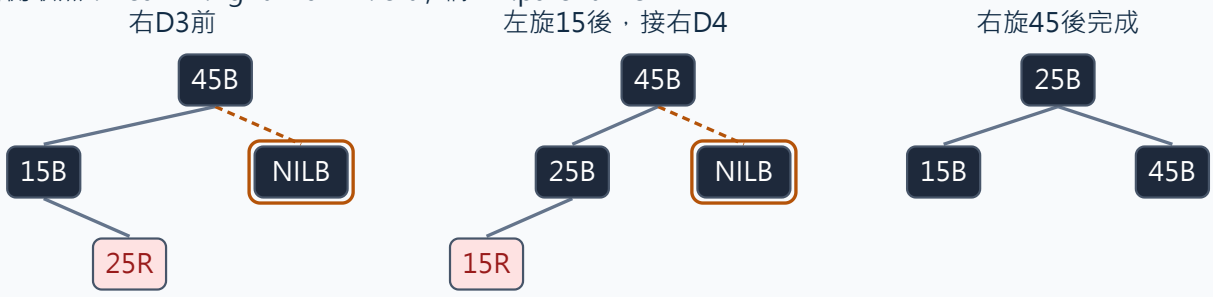
插入後root35B、左15B(right25R)、右45B。刪35時後繼y45B恰為z.right，x=NIL；y移成根後，x是45.right，必設x.parent=45，不能仍指舊35。

此時w15B在x左側，near=w.right=25R、far=w.left=NIL黑。右側D3把25黑、15紅，左旋15，使w25B、far15R。右側D4將25取45原黑、45黑、15黑，右旋45，最後25B(left15B,right45B)。

第13章 紅黑樹：詳解 （續）

右側D3 / D4：方向、near及far一起鏡射

右側缺黑：near=w.right，far=w.left；橘NIL.parent=45。



刪35的後繼45B直接移成根；最後25B(left15B,right45B)，record45保留。

45整個record從後繼位置移成局部根，再因旋轉成25.right；D45不變，35及D35被移除。若只換兄弟方向，卻把near誤認w.left=NIL，會把該NIL當紅near去重上色，或嘗試對沒有left的15右旋，違反rotation前提。

結論與檢核

鏡射必同時換child判斷、near/far、旋轉方向及parent接線；不能只換一個sibling欄位。

13-E10 高度界不是旋轉次數界

推導

bh(root)=3給$n \geq 2^3-1=7$，並直接給$h \leq 2 \cdot 3=6$；不能把只知$n \geq 7$錯當n=7代入一般上界約束任意更大樹。N=1023時一般$h \leq 2\log_2 1024=20$。單點查詢O(h)，完整走訪至少輸出1023點，Θ(n)。

插入只有I1會重複且不旋轉；一旦進I2會接I3並結束，最多2旋。刪除只有D2會上移且不旋；D1把p變紅，若接D2下一次因x紅退出，若接D3/D4也立即結束；故D1至多一次。D3/D4各一次，合計至多3旋。

結論與檢核

I1或D2仍可沿Θ(log n)祖先重新著色／計數，且先搜尋也需該高度。常數次旋轉不等於整個操作最壞常數時間。

第13章 紅黑樹：詳解 （續）

13-E11 Sentinel 與API的合法不變量

推導

(a)(c)(d)(e)都應驗；(b)不可能同時成立，因多個空位置共用同一NIL.parent。它可在刪除中暫存當前x位置的parent，旋轉其他空β不任意覆寫它。穩定時可回NIL，任何時候NIL.color必黑。

重複插入先在搜尋時找到相等key便回同一record，不創新紅葉、不改shape/color。找不到刪除key時直接NOT_FOUND，不進RB-DELETE。NIL的key無定義，若把它當普通z操作，TRANSPLANT或讀child會失去有效位置與物件身分，可能把root或sentinel父接壞。

結論與檢核

驗證parent只對真內部child要求雙向一致，NIL例外須按演算法上下文處理；不能因NIL特例就省掉普通節點parent檢查。

13-E12 旋轉單獨不保紅黑

推導

左旋20後root30R、left20B、20.left10R；中序仍10,20,30。Root紅違性質2。由30到右NIL只有NIL一黑；到20.right的NIL有20與NIL兩黑；到10下NIL也有20與NIL兩黑。所以性質5也失效。

只把root30塗黑，右路變2黑、左兩類路變3黑，仍不等。BST性質只要求鍵值區間與child序；它不含color或黑高要求，所以「旋轉保BST」與「旋轉不自動保紅黑」完全一致。

結論與檢核

合法紅黑修正必檢查所用case的色前提與黑數；不是在任何節點旋轉再把root塗黑就完成平衡。

第13章 紅黑樹：詳解 （續）

原創驗算程式附錄: red_black_checks.py

```
"""Red-black operation behavior, branch witnesses and stable record identities."""
from itertools import permutations
from random import Random
from math import log2


class Node:
    def __init__(self,key=None,color='B'):
        self.key=key;self.color=color;self.payload=('record',key)
        self.left=self.right=self.parent=None


class Tree:
    def __init__(self,capture=False):
        self.nil=Node();self.nil.left=self.nil.right=self.nil.parent=self.nil
        self.root=self.nil;self.capture=capture;self.events=[];self.rotations=0
    def snapshot(self):
        def visit(x):return None if x is self.nil else
(x.key,x.color,visit(x.left),visit(x.right))
        return visit(self.root)
    def event(self,code,x):
        if self.capture:self.events.append({'code':code,'tree':self.snapshot(),'focus':None
if x is self.nil else x.key,'parent':None if x.parent is self.nil else x.parent.key})
        else:self.events.append(code)
    def left_rotate(self,x):
        y=x.right;assert y is not self.nil;p=x.parent
        x.right=y.left
        if y.left is not self.nil:y.left.parent=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第13章 紅黑樹：詳解 （續）

```
        y.parent=p
        if p is self.nil:self.root=y
        elif x is p.left:p.left=y
        else:p.right=y
        y.left=x;x.parent=y;self.rotations+=1
    def right_rotate(self,x):
        y=x.left;assert y is not self.nil;p=x.parent
        x.left=y.right
        if y.right is not self.nil:y.right.parent=x
        y.parent=p
        if p is self.nil:self.root=y
        elif x is p.right:p.right=y
        else:p.left=y
        y.right=x;x.parent=y;self.rotations+=1
    def search(self,key):
        x=self.root
        while x is not self.nil and x.key!=key:x=x.left if key<x.key else x.right
        return x
    def insert(self,key):
        self.events=[];self.rotations=0;p=self.nil;x=self.root
        while x is not self.nil:
            p=x
            if key==x.key:return x
            x=x.left if key<x.key else x.right
        z=Node(key,'R');z.left=z.right=self.nil;z.parent=p
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第13章 紅黑樹：詳解 （續）

```
        if p is self.nil:self.root=z
        elif key<p.key:p.left=z
        else:p.right=z
        original=z;self.event('insert:leaf',z)
        while z.parent.color=='R':
            p=z.parent;g=p.parent;left=p is g.left;side='L' if left else 'R'
            uncle=g.right if left else g.left
            if uncle.color=='R':
                self.event('I-'+side+'1:before',z)
                p.color=uncle.color='B';g.color='R';z=g
                self.event('I-'+side+'1:after',z)
            else:
                if (z is p.right) if left else (z is p.left):
                    self.event('I-'+side+'2:before',z)
                    z=p
                    (self.left_rotate if left else self.right_rotate)(z)
                    self.event('I-'+side+'2:after',z)
                self.event('I-'+side+'3:before',z)
                z.parent.color='B';z.parent.parent.color='R'
                (self.right_rotate if left else self.left_rotate)(z.parent.parent)
                self.event('I-'+side+'3:after',z)
        self.root.color='B';self.root.parent=self.nil;self.nil.parent=self.nil
        self.event('insert:end',original);assert self.rotations<=2
        return original
    def transplant(self,u,v):
        if u.parent is self.nil:self.root=v
        elif u is u.parent.left:u.parent.left=v
        else:u.parent.right=v
        v.parent=u.parent
    def minimum(self,x):
        while x.left is not self.nil:x=x.left
        return x
    def delete(self,key):
        self.events=[];self.rotations=0;z=self.search(key)
        if z is self.nil:return False
        self.event('delete:start',z);y=z;color=y.color
        if z.left is self.nil:
            x=z.right;self.transplant(z,x)
        elif z.right is self.nil:
            x=z.left;self.transplant(z,x)
        else:
            y=self.minimum(z.right);color=y.color;x=y.right
            if y.parent is z:x.parent=y
            else:
                self.transplant(y,x);y.right=z.right;y.right.parent=y
            self.transplant(z,y);y.left=z.left;y.left.parent=y;y.color=z.color
        self.event('delete:splice',x)
        if color=='B':self.fix_delete(x)
        if self.root is not self.nil:self.root.parent=self.nil
        self.nil.parent=self.nil;z.left=z.right=z.parent=self.nil
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第13章 紅黑樹：詳解 （續）

```
        self.event('delete:end',self.root);assert self.rotations<=3
        return True
    def fix_delete(self,x):
        while x is not self.root and x.color=='B':
            p=x.parent;left=x is p.left;side='L' if left else 'R'
            w=p.right if left else p.left;assert w is not self.nil
            if w.color=='R':
                self.event('D-'+side+'1:before',x)
                w.color='B';p.color='R'
                (self.left_rotate if left else self.right_rotate)(p)
                w=x.parent.right if left else x.parent.left
                self.event('D-'+side+'1:after',x)
            near=w.left if left else w.right;far=w.right if left else w.left
            if near.color==far.color=='B':
                self.event('D-'+side+'2:before',x)
                w.color='R';x=x.parent
                self.event('D-'+side+'2:after',x)
            else:
                if far.color=='B':
                    self.event('D-'+side+'3:before',x)
                    near.color='B';w.color='R'
                    (self.right_rotate if left else self.left_rotate)(w)
                    w=x.parent.right if left else x.parent.left
                    self.event('D-'+side+'3:after',x)
                self.event('D-'+side+'4:before',x)
                w.color=x.parent.color;x.parent.color='B'
                (w.right if left else w.left).color='B'
                (self.left_rotate if left else self.right_rotate)(x.parent)
                x=self.root
                self.event('D-'+side+'4:after',x)
        x.color='B'
    def check(self,expected):
        assert self.nil.color=='B' and self.nil.left is self.nil and self.nil.right is
self.nil
        assert self.root.parent is self.nil and self.root.color=='B'
        seen={};keys=[]
        def visit(x,p,lo,hi):
            if x is self.nil:return 1,0
            assert id(x) not in seen;seen[id(x)]=x
            assert x.parent is p and lo<x.key<hi and x.color in ('R','B')
            assert x.payload==('record',x.key)
            if x.color=='R':assert x.left.color==x.right.color=='B'
            lb,lh=visit(x.left,x,lo,x.key);keys.append(x.key);rb,rh=visit(x.right,x,x.key,hi)
            assert lb==rb
            return lb+(x.color=='B'),1+max(lh,rh)
        bh,height=visit(self.root,self.nil,float('-inf'),float('inf'))
        assert keys==sorted(expected) and height<=2*log2(len(expected)+1)
        return {x.key:x for x in seen.values()},bh-1 if self.root is not self.nil else
0,height


def build(order,capture=False):
    t=Tree(capture)
```

第13章 紅黑樹：詳解 （續）

```
    for key in order:t.insert(key)
    return t


def main():
    witnesses={};insertions=deletions=0;coverage=set()
    def record(t,order,deleted=None):
        for code in t.events:
            if ':before' in code:
                k=code.split(':')[0];coverage.add(k)
                witnesses.setdefault(k,{'insert':list(order),'delete':deleted})
    for n in range(1,7):
        for order in permutations(range(1,n+1)):
            t=Tree();expected=set()
            for i,key in enumerate(order):

t.insert(key);expected.add(key);t.check(expected);record(t,order[:i+1]);insertions+=1
            for key in order:
                q=build(order);live=q.check(expected)[0];assert q.delete(key)
                now=q.check(expected-{key})[0]
                assert all(now[k] is node for k,node in live.items() if k!=key)
                record(q,order,key);deletions+=1
    rng=Random(13024);t=Tree();expected=set();mixed=0
    for _ in range(6000):
        key=rng.randrange(100)
        if rng.random()<.55:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第13章 紅黑樹：詳解 （續）

```
            before=t.search(key);node=t.insert(key)
            if before is not t.nil:assert node is before
            expected.add(key)
        else:
            old=key in expected;assert t.delete(key)==old;expected.discard(key)
        t.check(expected);mixed+=1
        coverage.update(code.split(':')[0] for code in t.events if ':before' in code)
    for key in sorted(expected,reverse=True):assert t.delete(key)
    t.check(set());assert t.root is t.nil and not t.delete(999)
    wanted={f'I-{s}{k}' for s in 'LR' for k in [1,2,3]}|{f'D-{s}{k}' for s in 'LR' for k in
[1,2,3,4]}
    assert wanted<=coverage,(wanted-coverage)
    # Explicit NIL-parent and successor-moving object paths, including direct and deep
successor.
    for order,key in [([40,20,60,10,30,50,70,45,55],40),([20,10,30,5,15,35],20),([1],1)]:
        t=build(order);before=t.check(set(order))[0];t.delete(key);after=t.check(set(order)-
{key})[0]
        assert all(after[k] is node for k,node in before.items() if k!=key)
    print(f'PASS insertion_prefixes={insertions} single_deletions={deletions}
mixed_operations={mixed} '
          'all14_mirrored_fixup_cases=True
root_NIL_global_BST_parent_blackheight_redchildren_payload_identities=True '
          'max_insert_rotations2_max_delete_rotations3=True')
    print('WITNESSES',witnesses)
    return witnesses


if __name__=='__main__':main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study